NT219 – Cryptography
Week 5: Modern Symmetric Ciphers (P2)
Updated for 2025

PhD. Ngoc-Tu Nguyen
tunn@uit.edu.vn

University of Information Technology, VNU-HCM

October 5, 2025

These slides update and extend the original Week 5 deck (AES, modes, and practice).
Focus: practical, secure-by-default choices for 2025 deployments.

Outline

1. AES Cipher
   - AES Cipher-Substitution
   - AES Cipher-ShiftRows
   - AES Cipher-MixColumns
   - AES Cipher-AddRoundKey
2. Modes of Operation
3. Implementation Guidance
4. Appendix
5. AES Cryptoanalysis
6. Best Known Single-Key Attacks (Classical)
7. Related-Key and Key-Schedule Notes
8. Operational Guidance (2025)

DES Review

- 64-bit block, 56-bit key; 16-round Feistel; S/P network.
- Now **insecure** to brute force; Triple-DES (3DES) is **deprecated** in new designs.
- Only keep as legacy knowledge; do *not* deploy.

AES (Rijndael) essentials

- Block size: 128 bits (state = 4×4 bytes).
- Key sizes: 128/192/256 bits; rounds = 10/12/14.
- Byte-oriented SPN (not Feistel).
- Round ops (enc): **SubBytes** (S), **ShiftRows**, **MixColumns**, **AddRoundKey**.
- Last round omits MixColumns.

GF(2^8) arithmetic
All byte ops are over $\mathrm{GF}(2^8)$ with irreducible poly
$x^8 + x^4 + x^3 + x + 1$.

MixColumns (enc) multiplies each column by $\begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix}$ over $\mathrm{GF}(2^8)$.

AES Operations

- **SubBytes**: nonlinear byte substitution via S-box defined from inversion in $\mathrm{GF}(2^8)$ followed by an affine transform.
- **ShiftRows**: row i of the state left-rotates by i bytes (i=0, 1, 2, 3) for diffusion.
- **MixColumns**: linear diffusion by multiplying each state column by a fixed MDS matrix over $\mathrm{GF}(2^8)$.
- **AddRoundKey**: XOR state with the round key; its own inverse.
- **Round structure**: SPN[1] with SubBytes→ShiftRows→MixColumns→AddRoundKey gives *confusion* & *diffusion* each round.

[1] Substitution-Permutation Network: Hệ mã dựa trên thay thế và chuyển vị

AES SubBytes overview (1/7)

- SubBytes introduces **nonlinearity** in AES.
- Each byte of the 4×4 state matrix is replaced independently by an S-box value.

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \Rightarrow \begin{bmatrix} S(s_{0,0}) & S(s_{0,1}) & S(s_{0,2}) & S(s_{0,3}) \\ S(s_{1,0}) & S(s_{1,1}) & S(s_{1,2}) & S(s_{1,3}) \\ S(s_{2,0}) & S(s_{2,1}) & S(s_{2,2}) & S(s_{2,3}) \\ S(s_{3,0}) & S(s_{3,1}) & S(s_{3,2}) & S(s_{3,3}) \end{bmatrix}$$

AES SubBytes Encryption (S-boxlook up table) (2/7)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

AES SubBytes Encryption (S-boxlook up table) (3/7)

- Example 1: $S(0x00) = 0x63$, and.
- Example 2: $S(0x53) = 0xED$ (via lookup).
- Example 3: $S(0x19) = 0xD4$, used in FIPS-197 example.

AES SubBytes Decryption (Invert S-boxlook up table) (4/7)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

AES SubBytes Decryption (Invert S-boxlook up table) (5/7)

- Example 4: $S(0x63) = 0x00$, and.
- Example 5: $S(0xED) = 0x53$ (via lookup).
- Example 6: $S(0xD4) = 0x19$, used in FIPS-197 example.

Mathematical Formation for Substitutions (Forward) (6/7)

Affine matrix (A):

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Forward SubBytes (S-box):

- Input byte: $a = (a_7 \dots a_0)_2$.
- Step 1: Compute multiplicative inverse $b = a^{-1}$ in $GF(2^8)$, with $b = 0$ if $a = 0$.
- Step 2: Apply affine transform

  $c = A \cdot b \oplus d, where\ d = (01100011)_2 = 0x63$

- Output (c = S(a)).

Example:

$$a = 0x53; \Rightarrow; b = 0xCA; \Rightarrow; c = 0xED$$

Mathematical Formation for Substitutions (Inverse) (7/7)

Inverse affine matrix (A^{-1}):

$$A^{-1} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

Inverse SubBytes (Inverse S-box):

- Input byte c.
- Step 1: Undo affine transform:

$$b = A^{-1} \cdot (c \oplus d).$$

- Step 2: Compute multiplicative inverse in $GF(2^8)$:

$$a = b^{-1}.$$

- Output $a = S^{-1}(c)$.

Example (inverse):

$$c = 0xED \;\Rightarrow\; b = 0xCA \;\Rightarrow\; a = 0x53$$

AES ShiftRows Transformation (1/2)

State Matrix Transformation:

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \Rightarrow \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

ShiftRows rule:

- Row 0: no shift.
- Row 1: left rotate by 1 byte.
- Row 2: left rotate by 2 bytes.
- Row 3: left rotate by 3 bytes.

Example:

$$\begin{bmatrix} 00 & 01 & 02 & 03 \\ 10 & 11 & 12 & 13 \\ 20 & 21 & 22 & 23 \\ 30 & 31 & 32 & 33 \end{bmatrix} \Rightarrow \begin{bmatrix} 00 & 01 & 02 & 03 \\ 11 & 12 & 13 & 10 \\ 22 & 23 & 20 & 21 \\ 33 & 30 & 31 & 32 \end{bmatrix}$$

AES Inverse ShiftRows Transformation (2/2)

Input: state after ShiftRows (encryption output)

$$\begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} \Rightarrow \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Inverse ShiftRows rule:

- Row 0: no shift.
- Row 1: right rotate by 1 byte.
- Row 2: right rotate by 2 bytes.
- Row 3: right rotate by 3 bytes.

Example:

$$\underbrace{\begin{bmatrix} 00 & 01 & 02 & 03 \\ 11 & 12 & 13 & 10 \\ 22 & 23 & 20 & 21 \\ 33 & 30 & 31 & 32 \end{bmatrix}}_{\text{state after ShiftRows}} \Rightarrow \begin{bmatrix} 00 & 01 & 02 & 03 \\ 10 & 11 & 12 & 13 \\ 20 & 21 & 22 & 23 \\ 30 & 31 & 32 & 33 \end{bmatrix}$$

AES MixColumns Transformation (1/2)

Matrix Formulation:

$$\begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \cdot \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Example (one column):

$$\begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix} \Rightarrow \begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix}$$

Multiplication in $GF(2^8)$ with polynomial $x^8 + x^4 + x^3 + x + 1$.

AES Inverse MixColumns Transformation (2/2)

Matrix Formulation:

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \cdot \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Example (one column):

$$\begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix} \Rightarrow \begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix}$$

Inverse constants: {0e, 0b, 0d, 09} in $GF(2^8)$.

AES AddRoundKey Transformation (1/3)

Definition:

- Each byte of the state matrix is XORed with the corresponding byte of the round key.
- This is the only step where the 128 bits round key directly enters the AES round.
- Same operation is used in both **encryption** and **decryption**.

$$\begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \oplus \begin{bmatrix} k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Example:

$$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix} \oplus \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix} = \begin{bmatrix} b9 & 28 & b9 & c3 \\ c7 & a0 & 65 & 94 \\ 1d & ce & b4 & 3e \\ a9 & 9a & 13 & 0d \end{bmatrix}$$

Note: $\oplus$ denotes XOR, identical in both encryption and decryption.

AES Key Schedule (2/3)

Goal: Expand a short cipher key into multiple round keys.

- AES supports key sizes of **128, 192, or 256 bits**.
- The key is expanded into an array of 32-bit words: $\{W[1], W[2], ...W[n-1]\}$.
- Number of rounds N_r and expanded words:
  - AES-128: $N_r = 10$, expanded key n = 44 words.
  - AES-192: $N_r = 12$, expanded key n = 52 words.
  - AES-256: $N_r = 14$, expanded key n = 60 words.
- Round keys = groups of 4 words (128 bits).

$$\text{Initial key} \Rightarrow \underbrace{\text{Expanded words}}_{\text{via nonlinear PRF-like steps}} \Rightarrow \text{Round keys}$$

Terminology: N_r = number of rounds. Expanded words depend on key length.

AES Key Schedule (3/3)

Expansion process:

- Input: cipher key of 128, 192, or 256 bits.
- Represented as k_{wlen} 32-bit words:

$$k_{\text{wlen}} = \begin{cases} 4 & \text{for 128-bit key} \\ 6 & \text{for 192-bit key} \\ 8 & \text{for 256-bit key} \end{cases}$$

- Output: expanded array $\{W[0], W[1], \ldots, W[n-1]\}$ of 32-bit words, where $n = 44, 52, 60$ depending on key size.
- Uses 3 core functions:
  - **RotWord:** cyclic shift of a 4-byte word.
  - **SubWord:** apply S-box to each byte.
  - **Rcon:** round constant $\{02^{i-1}, 00, 00, 00\}$ in $GF(2^8)$.

Pseudocode:

$$W[i] = \begin{cases} W[i-k_{\text{wlen}}] \oplus \text{SubWord}(\text{RotWord}(W[i-1])) \oplus \text{Rcon}[i/k_{\text{wlen}}], & i \equiv 0 \pmod{k_{\text{wlen}}} \\ W[i-k_{\text{wlen}}] \oplus \text{SubWord}(W[i-1]), & (k_{\text{wlen}} = 8) \wedge i \equiv 4 \pmod{k_{\text{w}}} \\ W[i-k_{\text{wlen}}] \oplus W[i-1], & \text{otherwise} \end{cases}$$

Round keys: consecutive 4-word groups → 128-bit round keys.

Modes of Operation: Taxonomy and status

- **Confidentiality-only (legacy)**: ECB, CBC, CFB, OFB, CTR.
- **Authenticated Encryption (AEAD)**: **GCM/GMAC**, **CCM**, **SIV families** (e.g., **AES-GCM-SIV**).
- **Storage only**: **XTS-AES** (no integrity, sector tweaked).
- **Format-Preserving Encryption (FPE)**: FF1, FF3-1 (specialized use).

Modes of Operation: ECB, CBC, CFB, OFB, CTR in 2025

Legacy/teaching:

- **ECB**: pattern leaks; do not use.
- **CBC**: fragile to padding oracles unless carefully composed with MAC; not parallelizable on enc.
- **CFB/OFB**: stream-like; rarely justified over modern AEAD.

Still useful:

- **CTR**: good building block; but must be paired with MAC/AEAD. Easy parallelism; careful counter management.

Modes of Operation: AEAD choices

- **AES-GCM**: fastest with AES-NI/ARMv8 Crypto Extensions; requires unique 96-bit nonces.
- **AES-GCM-SIV**: misuse-resistant (safe under nonce repetition with same key); deterministic per (key, nonce, AAD, PT).
- **CCM**: used in constrained environments (e.g., 802.15.4); longer authentication latency.

AEAD: General Structure

Authenticated Encryption with Associated Data (AEAD)
Ensures:
1. Confidentiality (encryption of plaintext).
2. Integrity authenticity (verification of ciphertext + AAD).

Inputs:

- K : secret key
- N : nonce/IV
- A : associated data (authenticated but not encrypted)
- P : plaintext

Outputs:

- C : ciphertext
- T : authentication tag

AES-CTR Mode: Pseudocode

AES-CTR: Counter Mode Encryption/Decryption

```
# Encryption
def AES_CTR_Encrypt(K, N, P):
    C = []
    for i, block in enumerate(split_blocks(P)):
        counter = N || int_to_bytes(i)   # Concatenate nonce and counter
        keystream = AES_Encrypt(K, counter)
        C_block = block ⊕ keystream
        C.append(C_block)
    return concat(C)
# Decryption
def AES_CTR_Decrypt(K, N, C):
    # Same as encryption (XOR with keystream)
    P = []
    for i, block in enumerate(split_blocks(C)):
        counter = N || int_to_bytes(i)
        keystream = AES_Encrypt(K, counter)
        P_block = block ⊕ keystream
        P.append(P_block)
```

AES-CCM: Pseudocode

```
# CCM = CTR + CBC-MAC
def AES_CCM_Encrypt(K, N, A, P):
    T = CBC_MAC(K, N, A, P)          # Compute authentication tag
    C = AES_CTR_Encrypt(K, N, P)
    return (C, T)

def AES_CCM_Decrypt(K, N, A, C, T):
    P = AES_CTR_Decrypt(K, N, C)
    T_check = CBC_MAC(K, N, A, P)
    if T_check != T:
        raise AuthenticationError
    return P
```

AES-GCM-SIV: Pseudocode

```
# Deterministic, misuse-resistant variant
def AES_GCM_SIV_Encrypt(K, N, A, P):
    S = POLYVAL(K, A || P)          # Compute synthetic IV
    C = AES_CTR_Encrypt(K, S, P)
    T = S
    return (C, T)

def AES_GCM_SIV_Decrypt(K, N, A, C, T):
    P = AES_CTR_Decrypt(K, T, C)
    S_check = POLYVAL(K, A || P)
    if S_check != T:
        raise AuthenticationError
    return P
```

AES-GCM: Pseudocode

```
# Encryption
def AES_GCM_Encrypt(K, N, A, P):
    H = AES_Encrypt(K, 0^128)          # GHASH key
    C = AES_CTR_Encrypt(K, N, P)       # CTR mode encryption
    T = GHASH(H, A, C) ⊕ AES_Encrypt(K, N)
    return (C, T)

# Decryption
def AES_GCM_Decrypt(K, N, A, C, T):
    H = AES_Encrypt(K, 0^128)
    P = AES_CTR_Decrypt(K, N, C)
    T_check = GHASH(H, A, C) ⊕ AES_Encrypt(K, N)
    if T_check != T:
        raise AuthenticationError
    return P
```

GHASH: Polynomial Hash for AES-GCM

Purpose: GHASH computes an authentication tag over ciphertext and AAD in AES-GCM. It operates in the finite field $GF(2^{128})$. **Inputs:**

- H : hash subkey ($H = \text{AES}_K(0^{128})$)
- A : associated data
- C : ciphertext

Output:

- T : authentication tag (before final $\oplus$ with encrypted nonce)

Pseudocode:

```
def GHASH(H, A, C):
    X = 0^128                    # initialize accumulator
    for block in split_blocks(A):
        X = (X ⊕ block) * H   # multiplication in GF(2^128)
    for block in split_blocks(C):
        X = (X ⊕ block) * H
    X = X ⊕ length_block        # XOR with concatenation of lengths of A and C
    return X
```

Modes of Operation: XTS-AES (storage) and FPE (specialized)

- **XTS-AES**: sector/"tweak"-based confidentiality for storage; *no integrity*. Use with filesystem-level authentication if tamper detection is required.
- **FPE (FF1, FF3-1)**: preserve format (e.g., PANs); strict domain/parameter constraints; *not* a general-purpose replacement for AEAD.

Modes of Operation: IV/Nonce rules (Critical!)

- **AES-GCM**: 96-bit (12-byte) *unique* nonce per key. Reuse is catastrophic (key stream and GHASH reuse).
- **AES-CTR/OFB**: never repeat nonce/counter; keep counters disjoint per key.
- **CCM**: unique nonce; no reuse per key; slightly slower than GCM.
- **SIV modes (e.g., GCM-SIV)**: tolerate *nonce reuse* (deterministic AEAD) but still require key hygiene.
- **CBC**: needs unpredictable IV; avoid for new protocols; padding-oracle class pitfalls.

Modes of Operation: Where we are in real protocols

- **TLS 1.3**: AEAD-only (*no* CBC/RC4/stream-cipher suites). Common: AES_128/256_GCM, `CHACHA20_POLY1305`.
- **SSH, QUIC, IPsec, WireGuard**: all use AEADs; `chacha20-poly1305` is widely deployed where AES-NI is unavailable.
- **Storage**: full-disk encryption uses **XTS-AES** plus separate authentication if tamper-detection is needed.

Do's and Don'ts

Do

- Use **AEADs** with authenticated decryption API (fail-closed on tag failure).
- Use vetted libraries: OpenSSL 3.x, libsodium, BoringSSL, Go's `crypto`, Rust `ring`/`aes-gcm`/`chacha20poly1305`.
- Generate nonces with counters or strong RNG; **never reuse** (except SIV).
- Zeroize keys, separate keys per purpose, rotate regularly.

Don't

- Build "encrypt-then-MAC" yourself unless you fully understand pitfalls.
- Reuse key+nonce; serialize counters across processes without coordination.
- Ignore side channels: use constant-time primitives; avoid secret-dependent branches/table lookups.

Labs (suggested exercises)

Lab A: AES-256-GCM with OpenSSL

1. Generate key: `openssl rand -out key.bin 32`; nonce: `openssl rand -out nonce.bin 12`.
2. Encrypt: `openssl enc -aes-256-gcm -K $(xxd -p key.bin) -iv $(xxd -p nonce.bin) -in msg.txt -out ct.bin -aad meta.bin -tag tag.bin`.
3. Decrypt and verify tag; flip a bit in `ct.bin` to observe failure.

Lab B: ChaCha20-Poly1305 (libsodium/Go/Rust): repeat A with ChaCha20-Poly1305; benchmark on ARM laptop vs x86 AES-NI.

Mathematical Details: Bytes in $GF(2^8)$ (Representation & Sum)

AES uses $GF(2^8) = \mathbb{F}_2[x]/\langle m(x)\rangle$ with

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad \leftrightarrow \quad \texttt{0x11B}.$$

Byte $\leftrightarrow$ polynomial. A byte $b = (b_7 b_6 \dots b_0)_2$ represents

$$b(x) = b_7 x^7 + b_6 x^6 + \cdots + b_1 x + b_0.$$

Field addition (bytewise XOR). Coefficients are in $\mathbb{F}_2$, so addition is XOR on bits:

$$a \oplus b \quad \leftrightarrow \quad a(x) + b(x) \pmod 2.$$

Example. $\texttt{0x57} = x^6 + x^4 + x^2 + x + 1$, $\quad \texttt{0x83} = x^7 + x + 1$. Then

$$\texttt{0x57} \oplus \texttt{0x83} = \texttt{0xD4}.$$

Note. No carries in $\oplus$; it is bitwise XOR.

Byte addition $\oplus$ and multiplication $\odot$ in $GF(2^8)$ (no xtime)

Let $GF(2^8) = \mathbb{F}_2[x]/\langle m(x)\rangle$ with $m(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B). Represent a byte $u \in \{0, \ldots, 255\}$ by $u(x) = \sum_{i=0}^{7} u_i x^i$.
Addition: $a \oplus b$ is bitwise XOR (no carry), i.e. $a(x) + b(x)$ over $\mathbb{F}_2$.

Multiply-by-x (unary operator). Define the one-bit logical left shift

$$\mathrm{sh}_1(u) = (u \ll 1) \bmod 256,$$

and the reduction operator

$$\rho(u) = \begin{cases} \mathrm{sh}_1(u), & \text{if } u \text{ has MSB 0,} \\ \mathrm{sh}_1(u) \ \oplus \ \texttt{0x1B}, & \text{if } u \text{ has MSB 1,} \end{cases}$$

which implements $(x \cdot u) \bmod m(x)$. For $k \geq 0$, write $\rho^{(k)}$ for k-fold application.

Full byte multiplication (binary operator). Write $b = \sum_{i=0}^{7} b_i 2^i$ with $b_i \in \{0, 1\}$ (equivalently $b(x) = \sum b_i x^i$). Then

$$\boxed{a \ \odot \ b \ = \ \bigoplus_{i=0}^{7} \left(b_i \cdot \rho^{(i)}(a)\right)}$$

tức là: với mỗi bit $b_i = 1$, cộng XOR hạng $\rho^{(i)}(a)$; nếu $b_i = 0$ thì bỏ qua.

Examples in $GF(2^8)$

Assume $m(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B). For a byte u, define

$$\mathrm{sh}_1(u) = (u \ll 1) \bmod 256, \quad \rho(u) = \begin{cases} \mathrm{sh}_1(u), & \text{if MSB}(u) = 0, \\ \mathrm{sh}_1(u) \ \oplus \ \texttt{0x1B}, & \text{if MSB}(u) = 1. \end{cases}$$

Then $\{02\} \odot u = \rho(u)$ and $\{03\} \odot u = \rho(u) \oplus u$.

Addition (XOR) example

$$\texttt{0x57} \oplus \texttt{0x83} = \texttt{0xD4}.$$

Multiply-by-x examples

$u = \texttt{0x87} = (1000\,0111)_2$:

$\mathrm{sh}_1(u) = \texttt{0x0E}$, MSB $= 1$

$\Rightarrow \ \rho(u) = \texttt{0x0E} \oplus \texttt{0x1B} = \texttt{0x15}.$

$u = \texttt{0x57} = (0101\,0111)_2$:

$\mathrm{sh}_1(u) = \texttt{0xAE}$, MSB $= 0$

$\Rightarrow \ \rho(u) = \texttt{0xAE}.$

Full byte multiply ($\odot$) — classic example

$$\texttt{0x57} \odot \texttt{0x83} = \texttt{0xC1}.$$

Sketch: write $b = \texttt{0x83} = 1 + x^1 \cdot 0 + \cdots + x^7$. Then $a \odot b = \bigoplus_{i:b_i=1} \rho^{(i)}(a) = \rho^{(0)}(a) \oplus \rho^{(7)}(a)$.

MixColumns — one column worked

Input column $[b_0, b_1, b_2, b_3] = [\texttt{d4}, \texttt{bf}, \texttt{5d}, 30]$.

Row 1: $r_0 = (\{02\} \odot b_0) \ \oplus \ (\{03\} \odot b_1) \ \oplus \ b_2 \ \oplus \ b_3$.

$\{02\} \odot \texttt{d4} = \rho(\texttt{d4}) = \texttt{b3}$ (MSB $= 1$: $\texttt{a8} \oplus \texttt{1b}$).

$\{03\} \odot \texttt{bf} = \rho(\texttt{bf}) \oplus \texttt{bf} = (\texttt{7e} \oplus \texttt{1b}) \oplus \texttt{bf} = \texttt{da}.$

$\Rightarrow \ r_0 = \texttt{b3} \oplus \texttt{da} \oplus \texttt{5d} \oplus 30 = 04.$

Further reading (primary sources)

- NIST SP 800-38A/B/C/D/E/F/G (modes, CMAC, CCM, GCM/GMAC, XTS, Key Wrap, FPE).
- RFC 8446 (TLS 1.3 AEAD-only ciphersuites).
- RFC 8439 (ChaCha20-Poly1305 AEAD).
- RFC 8452 (AES-GCM-SIV, nonce misuse-resistant).

AES: Design properties that frustrate cryptanalysis

- **Strong nonlinearity**: 8×8 S-box with low differential uniformity (max. 4) and high nonlinearity; no fixed points/linear structures.
- **High diffusion**: MDS MixColumns (branch number 5) + ShiftRows ⇒ many *active S-boxes* across rounds (exponential trail cost for differential/linear attacks).
- **Key schedule**: Adds round-dependent constants and bytewise substitution/rotation to avoid slide/symmetry effects (but see related-key notes for AES-256).
- **Round structure**: SPN with SubBytes→ShiftRows→MixColumns→AddRoundKey gives *confusion* & *diffusion* each round.

Biclique key-recovery (full-round) – still not practical

Biclique technique breaks the generic 2^k barrier *slightly* for full AES:

Variant	Rounds	Time (blocks)	Memory
AES-128	10	$\approx 2^{126.1}$	negligible
AES-192	12	$\approx 2^{189.7}$	negligible
AES-256	14	$\approx 2^{254.4}$	negligible

Notes.

- Data requirements are small; complexities remain *far above* feasibility.
- Security margin vs brute force is reduced by < 2 bits for AES-128; essentially academic.

Reduced-round landscape (very high level)

Techniques: differential, linear, impossible differential, truncated/integral (Square), boomerang/rectangle, meet-in-the-middle, algebraic.

- **Integral/Square**: effective on reduced rounds (e.g., up to ∼6–8 rds depending on variant/model).
- **Impossible differential / boomerang**: best known for several reduced-round settings; still below full rounds.
- **Algebraic**: SAT/Groebner formulations solve toy rounds; scaling to full rounds is infeasible with current methods.

Takeaway: no known classical attack threatens full-round AES in the single-key model.

Related-key results (theoretical) and practical relevance

- **AES-128/192**: no practical related-key attacks on full rounds known.
- **AES-256**: key schedule is weaker structurally; *theoretical* related-key attacks on the full cipher exist under strong oracle access and chosen related keys.
- **Practical impact**: Standard protocols do not expose related-key oracles (keys are random, independent); use robust KDFs and separate keys per purpose/nonce space.

Timing/cache (software) and EM/power (hardware)

Cache/timing attacks (software).

- Table-based S-box or T-tables → secret-dependent memory access → timing/cache leakage.[2]
- **Mitigation**: use constant-time, *instruction-level* AES (AES-NI / ARMv8 Crypto); or masked/bit-sliced S-box without table lookups.[3]

Power/EM DPA/CPA (hardware).

- Power/EM traces enable key extraction via correlations.[4]
- **Mitigation**: masking (domain-oriented), shuffling, hiding (noise/balancing), glitch resistance, hardened S-boxes, randomness discipline.[5]

Microarchitectural.

- Preempt speculative/Transient-exec leakage with constant-time code, fences where required, and side-channel safe libraries.[6]

[2] Tấn công thời gian / bộ nhớ đệm: khai thác sự khác biệt về thời gian truy cập hoặc trạng thái bộ nhớ đệm do truy cập dữ liệu phụ thuộc bí mật.

[3] Constant-time: mã thực hiện trong thời gian không phụ thuộc vào dữ liệu bí mật. AES-NI / ARMv8 Crypto: tập lệnh tăng tốc phần cứng cho AES. Masking: che giá trị bí mật bằng giá ngẫu nhiên. Bit-slicing: cài đặt AES bằng thao tác bit để tránh lookup bảng.

[4] Power/EM traces: bản ghi công suất/đức trường điện từ phát ra khi thiết bị xử lý; DPA/CPA: phương phân

Differential Fault Analysis (DFA) and countermeasures

DFA on AES.

- Single/few-byte faults injected late in the round can reveal key bytes using differential relations between correct/faulty ciphertexts.[7]
- Practical on insecure devices (smartcards, embedded) without countermeures.[8]

Mitigations.

- Redundant computation (round duplication / inverse check), infective computations, parity on state, tamper sensors, clock/voltage monitors.[9]
- Authenticate *every* ciphertext (AEAD) and reject on tag failure to kill fault oracle usefulness.[10]

[7] DFA (Phân tích sai lệch do lỗi): tấn công gây lỗi vật lý (glitch) vào trạng thái trong thuật toán để sinh ciphertext lỗi, sau đó phân tích hiệu giữa ciphertext đúng và lỗi để suy khoá.

[8] Smartcard/embedded: thiết bị nhúng có hạn chế về phần cứng và bảo vệ, dễ bị DFA nếu thiếu biện pháp phòng.

[9] Redundant computation: lặp lại phép tính và so sánh kết quả; infective computation: khi phát hiện lỗi thì làm nhiễu hoặc biến đổi đầu ra để vô hiệu hoá oracle lỗi; parity/state checks: kiểm tra chẵn lẻ/ checksum nội bộ; tamper sensors: cảm biến can thiệp; clock/voltage monitors: giám sát điện/clock.

[10] AEAD: chế độ mã hoá có xác thực (Authenticated Encryption with Associated Data); từ chối ciphertext không hợp lệ phá khả năng lợi dụng ciphertext lỗi để tấn công.

Datasets for AES Analysis with ML/DL (Side-Channel Focus)

Goal. Train/evaluate classifiers/regressors for key-byte recovery under leakage models (HW/HV), masking, desync, noise.

Name	Platform	Leakage/Masking	#Traces (typ.)
ASCAD (classic)	STM32 (8-bit AES)	Unmasked & masked variants	50k–200k
ASCADf (desync)	STM32	Masked + random desync/jitter	50k–200k
DPAContest v4	SASEBO-GII (FPGA AES)	Unmasked	∼100k+
CHES CTF Traces	Various boards	Mostly unmasked	10k–100k
ChipWhisperer CW308	STM32F3 / AVR	Unmasked / simple masks	user-collected
AES_RD / AES_HD sets	FPGA/MCU (varies)	Random delay / HD-style leakage	50k+

Recommended pipeline.

- *Labeling*: HW($S(p_i \oplus k)$) or HV models at SubBytes output; choose target byte i.
- *Preproc*: alignment (CPA peak, DTW), band-pass, downsample, Z-score; optional desync augmentation.
- *Models*: shallow CNN(1D) $\rightarrow$ ResNet(1D) $\rightarrow$ Transformer(1D) for profiling; NB/SVM/LR as baselines.
- *Metrics*: GE (guessing entropy), SR (success rate), key-rank curves vs #traces.
- *Splits*: strict profiling/validation/attack split; avoid trace overlap and scenario leakage.

Putting it together

- Prefer **AES-GCM** (with hardware crypto) or **ChaCha20-Poly1305**; consider **AES-GCM-SIV** if nonce management is risky.
- Enforce unique 96-bit nonces for GCM; use counters or a robust nonce-derivation scheme.
- Use **constant-time** AES (AES-NI/ARMv8) or side-channel-safe S-box code; avoid table lookups.
- Protect implementations against **faults**; validate tags before release of any plaintext.
- Separate keys per purpose; derive via domain-separated KDFs; rotate sensibly.

Questions?